Building Something That Knows How It Became Itself

by Jeremy Iglehart — June 10, 2026

I built a working software system today in three hours and eight minutes.

I will tell you what it does, how it came to be, and what I think it proves about a new way to build things alongside AI. But I want to start with the problem I have always had with systems — the one this whole effort is an answer to.

The Problem

Every system I have ever built has a graveyard of context around it.

Not the code. Not the architecture. The reasoning — the trail of decisions that explains why the system is shaped the way it is, why a particular fork was taken over another, what was tried and discarded, what the data revealed that changed the direction. That reasoning lives in chat logs, in memory, in the heads of the people who were in the room. Sometimes in a comment. Usually nowhere. It evaporates as a natural consequence of building.

This is not a new problem. It is the oldest problem in engineering. You come back to something six months later and the artifact is right there, but the thinking is gone. So you re-derive it. You make the same mistakes again at reduced speed because you are not starting from scratch but you are not starting from full context either. You are starting from the artifact plus reconstruction, and reconstruction is the enemy of understanding.

When you are building alongside an AI, this problem compounds. The AI has no persistent memory of the reasoning. Every session starts from whatever context you re-inject. The common solution is photographic memory — dump everything back in, every session, hope the model can reconstruct what mattered. But photographic memory is the wrong dream, and I want to explain precisely why.

What a Stratigraph Is — and Why Anyone Should Care

I want to answer this question carefully, because the idea sounds simple until you grasp what it actually fixes.

A Stratigraph is an event-driven, markdown-native knowledge architecture that applies event-sourcing not just to facts, but to meaning.

Let me unpack that.

Event-sourcing is a software architecture principle that says: never overwrite state, only append events. What happened is immutable. You can always reconstruct the current state by replaying the events from the beginning. This has been a standard practice in software systems for decades. It is not new.

What a Stratigraph adds is this: the same discipline applied to the conclusions you draw from those events. Not just what happened — what it meant, maintained as a first-class layer alongside the events. And when new evidence challenges your current conclusion, you do not overwrite it. You archive it — verbatim, with a note recording what challenged it and what replaced it. The old conclusion is not deleted. It becomes a stratum. You can read it. You can see how the thinking moved, and why.

The geological metaphor is precise. Stratigraphy is the science of reading Earth's history by reading its rock layers. Each layer is immutable. Nothing is destroyed. You drill down and the history is legible — not just what is there now, but the sequence of what was there before, what conditions produced each layer, how the record

evolved. A Stratigraph applies this to knowledge: not just what the system knows, but *how it came to know it, and how that knowing changed over time*.

The full stack looks like this:

```
EDA (event-driven architecture)   — immutable events, derived
state
+ Markdown                         — portable, human-readable,
no tool lock-in
+ Meaning as events                — conclusions versioned the
way facts are
= Stratigraph
```

The thing that is new is the middle layer — versioning meaning itself. EDA versions facts. Git versions bytes. ADRs (Architecture Decision Records) capture individual decisions in isolation, statically, without a concept of challenge or evolution. A Stratigraph versions understanding — the derived, synthesized read of what the facts mean — with a full lineage of how that understanding changed over time.

The bootstrap kit lives at: https://github.com/JeremyIglehart/stratigraph

It is a single README that tells any reader — human or AI — everything needed to operate a Stratigraph. Clone it, detach it from the origin, rename it, and it becomes its own thing. That is what I did today.

Why This Makes AI Memory Better

Here is the claim I want to make clearly, because it is the reason this matters beyond the engineering novelty.

I do not want my AI thinking partners to have photographic memory. I want them to have stratigraphic memory.

Photographic memory — perfect recall of every token, every state, every byte — is the industry's current goal. Vector stores, longer

context windows, retrieval systems: all of these are attempts to give AI more of what amounts to a bigger, faster hard drive. More pixels stored, faster retrieval. The assumption is that more recall equals better understanding.

It does not. Photographic memory recalls every pixel and understands nothing about how the picture changed. You can store everything and understand nothing. The fog does not go away because you have more fog stored at higher fidelity.

What I actually need from a thinking partner across many sessions — across days, weeks, months of work on the same system — is not perfect recall. It is how the thinking moved. What did we believe on day one? What challenged it? What replaced it, and why? What did we try that did not work, and what did that teach us? These are the questions that photographic memory cannot answer structurally. The answer is not in the raw token stream. It is in the strata.

Stratigraphic memory solves three problems that photographic memory cannot:

Hot lookups — what is the current understanding? (Same as photographic memory — both can answer this.)

Cold lookups — what did we decide six months ago and why? In a photographic system, you search the archive and get the raw artifact, stripped of the reasoning that produced it. In a Stratigraph, the conclusion is memoized — cached at the time it was written, in the context of the evidence that produced it, pinned against drift. You get the answer, not just the data.

Longitudinal questions — how did our understanding of X change over time? This is the question photographic memory structurally cannot answer, and that a Stratigraph answers natively. The strata are the answer. You read them the way a geologist reads rock: this is what we thought, this is what challenged it, this is what we think now, and this is the gap between them where the fault slipped.

There is something else that matters. When a Stratigraph is live from the first word — when recording begins before the idea is fully formed and the instrument is present for the thinking — the thinking

itself is different. Ideas evolve differently when a live, inspectable knowledge system is participating in the reasoning than when the same ideas are reconstructed afterward. The map drawn while you walk changes where you step. The map drawn after only records where you went.

These are not the same artifact. A Stratigraph insists on that distinction and makes it readable.

What I Built Today

The project is called Vitals Station.

The idea: my phone has access to a significant amount of health instrument data — heart rate, sleep stages, respiratory rate, blood oxygen saturation, movement, heart rate variability. An app called Health Auto Export can package all of it and send it to a REST endpoint on a schedule. I wanted to receive that data automatically, store it faithfully and immutably, and have it available as a clean, interpreted read for other systems — starting with a personal AI I work with called Karma, whose self-knowledge instrument (called Atmos) speaks in meteorological metaphors for internal states.

This is not technically complicated in isolation. Writing a server that accepts a POST request is a solved problem. What was interesting was the question underneath it: *what does it mean to store this data well*? How do you build a data pipeline that a future reader can trust — not just trust to run correctly, but trust to be honest about how it became what it is?

I was building a Stratigraph. I knew this from the start — that was the point. The question was whether the pattern would actually hold under the weight of a real project with real wire format surprises and real design decisions that needed to change in real time.

It did.

The code lives at: https://github.com/JeremyIglehart/vitals-station

How It Actually Went

We started before the idea was fully formed.

The first thing filed was a conception event — an honest record that recording had begun and the idea had not yet been named. Not a plan. Not a specification. The Stratigraph was present for the thinking before the thinking had a shape. That is the live path. The gold standard. The rarer one — it requires deciding to record before the idea has proven itself.

Then the idea was spoken. The system got a name. We had a design conversation — about what pattern the ingestion layer should follow, what the projection layer should look like, how the consumer relationship should be modeled, what the server should run on — and every real decision was committed to the event log before a line of code was written. Not just what we decided. Why. What the alternatives were. What was rejected, and the reasoning behind the rejection.

The TLS certificate decision is a good example of how this works in practice. The Health Auto Export app requires HTTPS. The server was running plain HTTP. Three options existed: plain HTTP (rejected — app requirement), self-signed certificate (viable but requires a manual trust profile install on the iPhone, friction on every cert rotation), or Tailscale-issued Let's Encrypt certificate (chosen — free on the personal plan, trusted automatically by iOS, scoped to the private Tailscale network). That decision and its reasoning are in the genome. Six months from now, if someone wonders why the cert is provisioned the way it is, the answer is there. Not in a comment. In the strata.

The first real export arrived from the iPhone. 9.9 megabytes, a full day's instrument readings. The wire format differed from the documentation in small but meaningful ways — as wire formats always do. Three distinct data-point shapes where the documentation implied one. Heart rate arrives as a range (min, average, max, context) with no single value field — a special case. Sleep analysis

carries stage names as a value field alongside the duration quantity. Six high- frequency metrics arrive at per-second Watch sampling granularity — tens of thousands of near-identical micro-values per day.

That last one broke the first version of the anomaly detection system.

I had designed anomaly detection to flag readings more than two standard deviations from the day's mean — a standard statistical approach. When the first projections were built, the anomaly section was hundreds of lines of noise. Step count arrives as over twenty thousand nearly-identical micro-values per day. The standard deviation across them is infinitesimal. Almost every value is "statistically unusual" relative to the near-identical others — because the unit of measurement is a fraction of a step, not a step. The approach was wrong for this class of data.

The fix was to restrict anomaly detection to physiological signals only — heart rate, heart rate variability, blood oxygen, respiratory rate, body temperature. These are the signals where a genuine spike means something. The accumulator metrics (steps, distance, active energy) belong in daily totals, not anomaly flags. That decision, and the evidence that produced it, is in the event log.

The redesign took twenty minutes. The reasoning is permanently in the record. A future session — or a future collaborator — will not have to rediscover why the anomaly detection is scoped the way it is. They will find the stratum that explains it and the wire format data that demanded the change.

By the end of the session:

- A working TLS-secured REST endpoint on a private Tailscale network, receiving exports from the iPhone - A full ingestion pipeline with inbox/pending and inbox/processed directories, YAML provenance headers on every processed file, immutable event records, and automatic projection rebuild on every ingest - Three projection files at different temporal scales — micro (current read + today), meso (yesterday arc + three-day rolling window with timestamped anomalies), and macro (seven-day rolling window with

daily totals) - Anomaly detection on physiological signals, timestamped to the minute, ready for cross-reference against Atmos weather events - A Telegram notification to a private channel when each ingest completes, including any anomalies from the prior day - A systemd user service that runs the server independently of any AI session - A private GitHub repository — made public once real health data was removed from the full git history using filter-repo - An examples directory with fictional data showing the complete pipeline without exposing anything real - 25 commits, each one honest about what it represents

Three hours and eight minutes, start to finish.

What the Stratigraph Actually Did

I want to be specific about what the pattern contributed, because it is easy to say the word and miss the mechanism.

The Stratigraph did not just document decisions after the fact. It participated in making them.

When the anomaly detection failed on real data, the event log was what let us say precisely: here is what the wire format revealed, here is what that means for the approach, here is what changes and why. The conclusion was updated. The old one was archived, not lost. The reasoning chain is intact and readable.

When the question of how many days to load for a rolling-window projection came up, the answer came from the seam analysis — actual evidence from the real export data showing that daily exports regularly carry records from the prior day at the boundary. A strict seven-day window would silently drop those records. Load eight days of files. One extra file, one boundary day of buffer. That evidence is in the record. The reasoning is traceable. A collaborator reading this six months from now will find the exact data point that produced the decision.

And there is a harder thing to name — a quality of the work itself. When the system knows how it became itself, when every real decision has a home and a provenance trail, the thinking changes. You do not re-derive. You do not lose the thread. The instrument is present for the thinking and participates in it. That changes what gets built.

The map drawn while walking and the map drawn after are not the same artifact. I know this abstractly. I experienced it concretely today.

What This Is For

Any system that needs to remember not just what it knows but how it came to know it.

Research. Personal informatics. Organizational knowledge management. Long-running software projects where the reasoning around design decisions matters as much as the decisions themselves. Any context where AI is a sustained thinking partner across many sessions and you need the AI's understanding to stratify — to hold the layers of how it came to know what it knows — rather than flatten.

The minimum surface area is small. A README, an events directory, a conclusions file, and the discipline to write to them as the thinking happens. The payoff is a record that can be read by anyone — human or model — and from which the full history of the thinking is legible. Cold start, fresh session, new collaborator: the Stratigraph orients them without reconstruction.

The bootstrap kit is one file. Clone it, name it, and it becomes whatever you are building. The genome it carries is sufficient to operate it.

This is stratigraphic memory. Not photographic — every pixel stored, nothing understood. Stratigraphic — the layers held, the evolution legible, the strata readable on demand.

That is the difference between a hard drive and a mind.

Stratigraph bootstrap: https://github.com/JeremyIglehart/stratigraph
Vitals Station: https://github.com/JeremyIglehart/vitals-station
Session duration: 3 hours, 8 minutes. Commits: 25.